Plan de Idei și Barem de Lucru

Pentru Temele de Proiect Prolog

4 martie 2026

Cuprins

1 Introducere **2**

2 Considerații Generale **2**

2.1 Cerințe Comune pentru Toate Temele 2

3 Temele de proiect **3**

4 Cerințe de Predare **10**

1 Introducere

Acest document prezintă planul de idei și baremul de lucru pentru temele de proiect la laboratorul de Prolog. Fiecare temă este detaliată cu privire la abordarea recomandată și criteriile de evaluare. Punctajul maxim pentru fiecare proiect este de **70 de puncte**.

2 Considerații Generale

2.1 Cerințe Comune pentru Toate Temele

- **Documentație minimă:**
  - Fișier `README` cu descrierea problemei și a soluției implementate
  - Comentarii în cod pentru predicatele importante
  - Exemple de rulare cu output
  - Specificarea versiunii de Prolog utilizate
- **Stil de programare:**
  - Nume sugestive pentru predicate și variabile
  - Cod modular și reutilizabil
  - Gestionarea corectă a backtracking-ului
  - Evitarea cut-urilor inutile
- **Testare:**
  - Minimum 3 teste pentru fiecare predicat principal
  - Tratarea cazurilor limită
  - Verificarea corectitudinii rezultatelor

3 Temele de proiect

Tema 1: Problema Canibalilor

Enunț

Trei misionari și trei canibali se află la marginea unui râu, cu scopul de a trece pe celălalt mal. Au la dispoziție o barcă de două persoane. Dacă la un moment dat, pe un mal numărul canibalilor întrece pe cel al misionarilor, misionarii sunt în pericol. Problema constă în a afla cum pot trece râul cele 6 persoane în deplină siguranță.

Plan de Implementare

1. **Reprezentarea stării**
   - Definirea structurii:
     `stare(NCanibaliStg, NCanibaliDr, NMisionariStg, NMisionariDr, PozitieBarca)`
   - Pozitia bărcii: `stg` sau `dr`
2. **Validarea stării**
   - Predicat `in_siguranta/1` - verifică pentru ambele maluri că numărul misionarilor (dacă există) este mai mare sau egal cu numărul canibalilor
3. **Generarea mutărilor posibile**
   - Toate combinațiile de 1-2 persoane care pot traversa (canibal(i), misionar(i), sau amestec)
   - Predicat `traverseaza/2` - aplică mutarea și verifică starea rezultată
4. **Algoritmul de căutare**
   - DFS (Depth First Search) cu evitarea ciclurilor
   - Predicat `traseu/3` pentru găsirea soluției
5. **Afișarea rezultatului**
   - Formatare prietenoasă a pașilor

Barem de Lucru

Cerință	**Punctaj**
Definirea corectă a structurii de stare	10p
Implementarea predicatului `in_siguranta/1`	15p
Implementarea predicatului `traverseaza/2`	20p
Generarea tuturor mutărilor posibile	15p
Implementarea DFS cu evitarea ciclurilor	20p
Predicatul `traseu/3` funcțional	10p
Documentație și stil de cod	5p
Progres pe parcurs (versionare cod)	5p
Total	**100p**

Tabela 1: Barem de notare - Problema Canibalilor

Tema 2: Lupul, Capra și Varza

Enunț

Un țăran trebuie să treacă peste un râu un lup, o capră și o varză. Barca poate transporta doar țăranul și unul dintre cele trei elemente. Lăsate nesupravegheate, lupul mănâncă capra, iar capra mănâncă varza. Să se găsească o soluție pentru a trece toate elementele în siguranță.

Plan de Implementare

1. **Reprezentarea stării**
   - `stare(MalTaran, MalLup, MalCapra, MalVarza)`
   - Malurile: `stg` sau `dr`
2. **Validarea stării**
   - Verificare că nu sunt lăsate împreună perechi problematice:
     - Lup cu capra fără țăran
     - Capra cu varza fără țăran
3. **Mutări posibile**
   - Țăranul traversează singur
   - Țăranul traversează cu unul dintre cele 3 elemente
4. **Căutarea soluției**
   - Implementare BFS sau DFS cu detecția stărilor de succes

Barem de Lucru

Cerință	**Punctaj**
Definirea stării problemei	10p
Predicat de validare a stării	20p
Generarea mutărilor posibile	20p
Implementarea algoritmului de căutare	20p
Evitarea buclelor infinite	15p
Afișarea soluției	5p
Documentație	5p
Progres pe parcurs (versionare cod)	5p
Total	**100p**

Tabela 2: Barem de notare - Lupul, Capra și Varza

Tema 3: Enigma lui Einstein

Există cinci case, fiecare cu cinci atribute: culoare, naționalitate, băutură, țigară, animal. Pe baza celor 14 indicii date, trebuie să se deducă cine bea apă și cine deține zebra.

1. Englezul locuiește în casa roșie.
2. Spaniolul deține câinele.
3. Se bea cafea în casa verde.
4. Ucraineanul bea ceai.
5. Casa verde este imediat la dreapta casei de culoarea fildeșului.
6. Fumătorul de Old Gold deține melci.
7. Se fumează Kools în casa galbenă.
8. Se bea lapte în casa din mijloc.
9. Norvegianul locuiește în prima casă.
10. Omul care fumează Chesterfields locuiește în casa vecină cu cel care are vulpea.
11. Se fumează Kools în casa vecină cu cea în care este ținut calul.
12. Fumătorul de Lucky Strike bea suc de portocale.
13. Japonezul fumează Parliaments.
14. Norvegianul locuiește lângă casa albastră.

Plan de Implementare

1. **Reprezentarea domeniului**
   - 5 case, fiecare cu atributele: `casa(Culoare, Nationalitate, Bautura, Tigara, Animal)`
   - Definirea listelor de valori posibile pentru fiecare atribut
2. **Implementarea constrângerilor**
   - Fiecare dintre cele 14 enunțuri devine o regulă
   - Constrângeri de poziționare: `langa/2`, `imediat_la_dreapta/2`
3. **Generarea soluției**
   - Predicat `enigme/1` care construiește lista celor 5 case
   - Backtracking pentru a satisface toate constrângerile simultan
4. **Extragerea răspunsului**
   - Identificarea casei unde se bea apă
   - Identificarea casei cu zebra

Barem de Lucru

Cerință	**Punctaj**
Definirea structurii casei și a domeniilor	10p
Implementarea constrângerilor 1-7	25p
Implementarea constrângerilor 8-14	25p
Generarea completă a soluției	15p
Identificarea corectă a răspunsurilor	15p
Documentație	5p
Progres pe parcurs (versionare cod)	5p
Total	**100p**

Tabela 3: Barem de notare - Enigma lui Einstein

Tema 4 și 5: Generarea unui Limbaj (Română/Engleză)

Enunț

Pornind de la o gramatică (simboluri terminale, neterminale, reguli de producție și simbol de start), să se genereze exprimări corecte gramatical în limba română sau engleză.

Plan de Implementare

1. **Definirea gramaticii**
   - Simboluri terminale (cuvinte)
   - Simboluri neterminale (propoziție, sintagmă nominală, sintagmă verbală etc.)
   - Reguli de producție de forma: `neterminal --> [terminal1, terminal2, ...]`
2. **Structura propoziției**
   - Subiect + Predicat + Complemente (opțional)
   - Acord gramatical (număr, persoană, gen)
3. **Generarea propozițiilor**
   - Predicat `genereaza/1` - generează o propoziție validă
   - Posibilitate de generare random sau sistematică
4. **Verificarea corectitudinii**
   - Predicat `propozitie/1` - verifică dacă o listă de cuvinte este o propoziție validă

Barem de Lucru

Cerință	**Punctaj**
Definirea terminalilor și neterminalilor	10p
Implementarea regulilor gramaticale de bază	20p
Acordul gramatical (gen, număr, persoană)	20p
Predicat de generare a propozițiilor	20p
Predicat de verificare	10p
Varietatea propozițiilor generate	10p
Documentație	5p
Progres pe parcurs (versionare cod)	5p
Total	**100p**

Tabela 4: Barem de notare - Generarea unui limbaj

Tema 6: Sistem Expert - Recomandare Carieră

Enunț

Dezvoltarea unui sistem expert care recomandă cariere pe baza abilităților, intereselor și obiectivelor profesionale ale utilizatorului.

Plan de Implementare

1. **Baza de cunoștințe**
   - `cariera(Nume, Domeniu, Industrie)` - informații generale
   - `abilitati(Cariera, ListaAbilitati)` - abilități necesare
   - `interese(Cariera, ListaInterese)` - interese asociate
   - `educatie(Cariera, NivelStudii)` - studii necesare
   - `salariu(Cariera, IntervalSalarial)` - interval salarial
2. **Reguli de potrivire**
   - Calcularea gradului de potrivire între abilitățile utilizatorului și cele cerute
   - Calcularea gradului de potrivire între interesele utilizatorului și cele asociate
3. **Sistem de scor**
   - Formula de calcul:
     `Scor = pondere_abilitati * match_abilitati + pondere_interese * match_interese`
   - Ierarhizarea recomandărilor
4. **Interfața cu utilizatorul**
   - Predicat `recomanda(AbilitatiUtilizator, IntereseUtilizator, Recomandari)`
   - Posibilitatea de filtrare după criterii suplimentare

Barem de Lucru

Cerință	**Punctaj**
Construirea bazei de cunoștințe (minim 10 cariere)	20p
Reguli de potrivire pe baza abilităților	15p
Reguli de potrivire pe baza intereselor	15p
Implementarea sistemului de scor	20p
Predicat principal de recomandare	10p
Gestionarea cazurilor fără potrivire	5p
Documentație și exemple	10p
Progres pe parcurs (versionare cod)	5p
Total	**100p**

Tabela 5: Barem de notare - Recomandare Carieră

Tema 7: Sistem Expert - Recomandare Rețetă Culinară

Enunț

Dezvoltarea unui sistem expert care recomandă rețete culinare pe baza ingredientelor disponibile, preferințelor utilizatorului și restricțiilor dietetice.

Plan de Implementare

1. **Baza de cunoștințe**
   - `ingredient(Nume)` - ingrediente disponibile în general
   - `reteta(Nume, ListaIngrediente)` - rețete și ingredientele necesare
   - `categorie(Reteta, Categorie)` - mic dejun, prânz, desert etc.
   - `timp_preparare(Reteta, Timp)` - timp în minute
   - `dificultate(Reteta, Dificultate)` - nivel de dificultate
   - `restrictie(Reteta, Restrictie)` - vegetarian, fără gluten etc.
2. **Reguli de recomandare**
   - Pe baza ingredientelor disponibile
   - Pe baza preferințelor (categorie, timp, dificultate)
   - Pe baza restricțiilor alimentare
3. **Optimizarea recomandărilor**
   - Prioritate pentru rețete care folosesc cât mai multe ingrediente disponibile
   - Minimizarea ingredientelor lipsă

Barem de Lucru

Cerință	**Punctaj**
Baza de cunoștințe (minim 15 rețete, 30 ingrediente)	20p
Predicat `recomanda_reteta/2` (ingrediente disponibile)	20p
Predicat `recomanda_reteta_cu_preferinte/3`	20p
Gestionarea restricțiilor alimentare	10p
Afișarea prietenoasă a rezultatelor	10p
Tratarea cazurilor speciale	5p
Documentație	10p
Progres pe parcurs (versionare cod)	5p
Total	**100p**

Tabela 6: Barem de notare - Recomandare Rețetă Culinară

Tema 8: Sistem Expert - Recomandare Ținute Vestimentare

Enunț

Dezvoltarea unui sistem expert care recomandă ținute vestimentare în funcție de vreme, eveniment, stil personal și preferințe.

Plan de Implementare

1. **Baza de cunoștințe**
   - `haina(Nume, Categorie, Sezon)` - tipuri de haine
   - `stil(NumeHaina, Stil)` - stilul articolului (casual, formal etc.)
   - `culoare(NumeHaina, Culoare)` - culori disponibile
   - `ocazie(Stil, Ocazie)` - ce ocazii se potrivesc cu un stil
   - `temperatura(Sezon, TempMin, TempMax)` - temperaturi specifice sezonului
2. **Factorii de decizie**
   - Vremea (temperatură, precipitații)
   - Evenimentul (formal, casual, sport)
   - Preferințe personale (culori, stil)
   - Sezon
3. **Reguli de combinare**
   - Principii de asortare (culori complementare)
   - Stratificare corectă (lenjerie, îmbrăcăminte de bază, outerwear)
   - Coerența stilului
4. **Generarea ținutei complete**
   - Recomandare ansamblu (top, bottom, încălțăminte, accesorii)

Barem de Lucru

Cerință	**Punctaj**
Baza de cunoștințe variată (minim 30 articole)	20p
Reguli pentru recomandare în funcție de vreme	15p
Reguli pentru recomandare în funcție de eveniment	15p
Implementarea principiilor de asortare	15p
Predicat principal `recomanda_tinuta/3`	10p
Generarea de ținute complete	10p
Documentație și exemple	10p
Progres pe parcurs (versionare cod)	5p
Total	**100p**

Tabela 7: Barem de notare - Recomandare Ținute Vestimentare

4 Cerințe de Predare

Structura Arhivei

- **/cod** - toate fișierele sursă (.pl)
- **/doc** - documentația proiectului
- **/teste** - exemple de test și rezultate
- **README.txt** - descriere sumară și instrucțiuni de rulare

Termen Limită Data limită pentru predarea proiectelor: **Săptămâna 5**, seara precendentă laboratorului, **ora 20:00**.

Modalitate de Predare

- Arhivă `ZIP` se încarcă pe platforma elarning, în cadrul temei de lucru *Proiect Prolog*
- Denumirea arhivei va conține numele și prenumele fiecărui membru al echipei, conform exemplului: `[Prolog] Proiect - Nume Prenume - Grupa`
- Prezentare orală obligatorie în cadrul laboratorului din Săptămâna 5 .

Sistem de Evaluare Peer-Review

- **Penalizări:**
  - Cod necomentat: -3p
  - Lipsa exemplelor de test: -3p
  - Plagiat: descalificare

Formula de Calcul

$$\text{Nota finală} = \underbrace{\frac{P_{\text{profesor}}}{10} \times 0.70}_{70\%} + \underbrace{M_{\text{feedback}} \times 0.15}_{15\%} + \underbrace{P_{\text{completare}}}_{15\%}$$

unde:

- P_{profesor} = punctajul acordat de profesor (maxim 70 de puncte)
- M_{feedback} = media punctajelor primite de la colegi în feedback (maxim 15 puncte)
- $P_{\text{completare}}$ = punctajul acordat în funcție de numărul de feedback-uri oferite (maxim 15 puncte)

Nr. formulare completate	**Punctaj ($P_{\text{completare}}$)**
8	15 puncte
7	13 puncte
6	11 puncte
5	9 puncte
4	7 puncte
< 4	0 puncte

Tabela 8: Punctajul acordat în funcție de numărul de evaluări completate